

DR-XAI Platform – Security & Functional Rebuild

Audit and targeted reconstruction of the doctor-facing web platform for diabetic retinopathy screening. Covers credential exposure remediation, PHI access control, restoration of broken clinical features, and the interface contract for the grading model still to be delivered.

DESIGNATION DR-XAI Platform	COMPONENT Web application (clinical)	DATE 14 September 2026
BASELINE 437ad4e	HEAD f7b58d1	COMMITTS 5
FILES AUDITED 62	FINDINGS 66	DEPLOYMENT TARGET Local demonstration
CLINICAL STATUS Not for clinical use		

01 Context

The application was generated by an AI coding tool and had never been reviewed. A full audit of all 62 source files returned 66 findings, 12 of them critical. Two were structural rather than incidental, and together they determined the shape of the work.

FINDING 1 — LIVE CREDENTIAL EXPOSURE

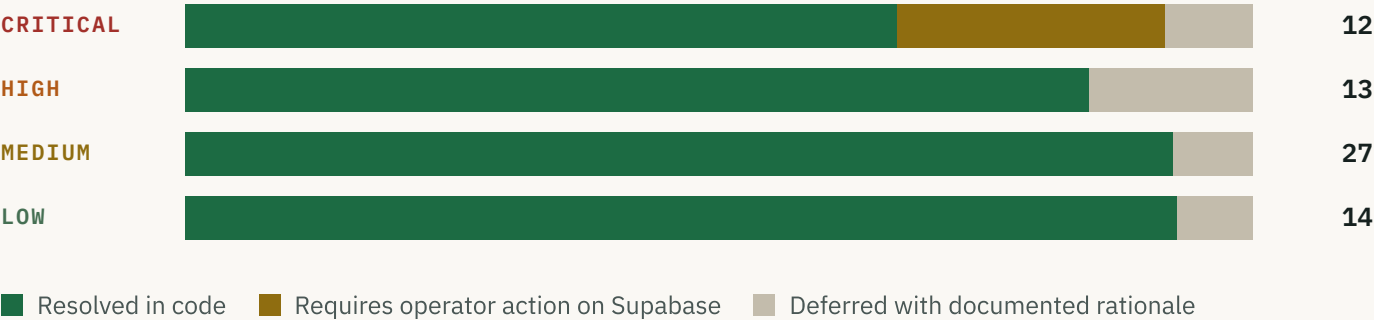
backend/.env contained a working Supabase PostgreSQL password, project URL and anonymous JWT key. The project had no .gitignore at any level and was not yet a Git repository, so the first commit would have published those credentials permanently. The directory is additionally synchronised to OneDrive.

FINDING 2 — CONTRACT MISMATCH WITH THE RESEARCH

The interface is built around an ICDR severity grade (stages 0–4) with a confidence score. Examination of the associated research repositories confirmed that no grading model exists: all four architectures perform lesion segmentation only, producing four-channel EX / HE / MA / SE masks. `model_service.py` returned `random.randint(0, 4)`. A doctor uploading a scan received a randomly chosen stage presented with a confidence bar, a clinical description and a downloadable signed report.

The grading model is owned by a separate developer and was out of scope. The platform was therefore rebuilt to be correct, secure and explicitly honest about producing simulated output, with a documented interface for that model to be connected later without further changes to the application.

02 Disposition of findings



Items marked *requires operator action* are complete in code but depend on steps only the account holder can perform: credential rotation, deletion of the seeded accounts, and creation of a private storage bucket. Deferred items are listed with their rationale in section 06.

03 Principal corrections

SECRETS AND CONFIGURATION

BEFORE	AFTER
SECRET_KEY defaulted to the literal string "changeme" when the environment variable was absent. The failure was silent.	Required field with no default; known placeholder values and any key shorter than 32 characters are rejected at startup. The application refuses to boot rather than sign tokens with a guessable key.

BEFORE

No `.gitignore` anywhere; no `.env.example`; no version control.

AFTER

Repository initialised behind a `.gitignore` excluding secrets, databases and `backend/uploads/`. Exclusion of each was verified before the first commit. A documented `.env.example` ships in its place.

PROTECTED HEALTH INFORMATION

BEFORE

Retinal scans were stored with `get_public_url()` and additionally served by an unauthenticated `StaticFiles` mount at `/uploads`. Anyone holding a URL could retrieve patient imaging indefinitely.

AFTER

Private bucket; opaque storage keys never exposed in API responses; the static mount removed. Images are served only by `GET /images/{diagnosis_id}`, which verifies ownership and returns 404 — not 403 — so the response does not confirm that an identifier exists.

BEFORE

No server-side validation of uploads. The client's `Content-Type` was trusted and the stored filename extension was taken from the client. No size limit.

AFTER

Uploads are validated by decoding them, restricted to five raster formats, capped at 10 MB by chunked read, and re-encoded to PNG — which is pixel-lossless but discards EXIF, where clinical devices record serial numbers and sometimes patient identifiers. Stored names are generated server-side.

AUTHENTICATION

BEFORE

Failed login returned 400, so the frontend's 401 interceptor never fired. No rate limiting existed anywhere in the application. Seeded accounts used `admin123` and `doctor123`, printed to the console and pushed to the live database.

AFTER

401 with a uniform message for both unknown-email and wrong-password cases, and a dummy hash verification on the unknown-email path so response timing cannot be used to enumerate registered addresses. Login is rate limited to five attempts per minute. Seeding takes its password from the environment, enforces a twelve-character minimum and refuses to run in production.

CLINICAL FEATURES THAT DID NOT WORK

BEFORE

The result page composed `"http://localhost:8000" + image_url`, where

AFTER

Images load through the authenticated route as a blob, since an `<img src>` cannot carry a bearer

<code>image_url</code> was already absolute. The retinal scan was broken on every diagnosis ever produced — the doctor was shown a verdict with no image against which to check it.	token. Verified rendering end to end.
BEFORE The doctor dashboard derived its figures from a patient-list request. Three of its four indicators were pinned at zero and the distribution chart permanently reported no data.	AFTER New <code>GET /stats/me</code> endpoint returning real per-doctor counts, severity distribution and a 30-day window.
BEFORE The reports page issued one request per patient inside a sequential loop — 101 round trips for 100 patients — with an empty catch block discarding every failure.	AFTER A single paginated <code>GET /diagnosis/</code> with the patient name joined server-side.
BEFORE <code>SystemLog</code> was defined and read by the administration page but never written. There was no record of who had accessed which patient record.	AFTER Audit service recording authentication, patient record access, diagnosis runs and administrative changes — a regulatory requirement for a system handling medical data, not a convenience.
BEFORE Administrative password reset accepted a new password, discarded it, and returned 200. An administrator unlocking a doctor saw success and nothing happened.	AFTER Applied and recorded. Request schemas were narrowed to exactly the fields each endpoint acts on, so silent discard is no longer representable.

TRACEABILITY OF RESULTS

Every diagnosis row now records `is_simulated`, `model_name` and `model_version`. The column is not nullable and defaults to `true` at database level, so an omission fails toward warning rather than toward silence. This is the single most consequential schema change: when a real model is connected, results produced during the simulated period remain permanently distinguishable instead of blending into genuine predictions.

Simulated results carry a warning banner in the interface, a badge on each row, and both a diagonal watermark and a header notice on the exported PDF — the artefact most likely to be printed, filed or forwarded and read without surrounding context.

INTERFACE QUALITY

- Eleven failure paths that logged only to the browser console now surface a message and a retry. Previously a failed request left an indefinite spinner or an empty table.
- Severity badges moved to dark text on light amber and orange. The previous white-on-yellow measured approximately 2.0:1 against the WCAG AA requirement of 4.5:1, and severity is clinical information.
- The fixed 240 px sidebar became a drawer below the 1g breakpoint; the content offset had been 256 px, leaving a misaligned gutter on every page.
- Request cancellation added to all data loading: rapid pagination could previously allow an earlier response to resolve last and overwrite current rows.
- Dialogs gained focus trapping, Escape and click-outside dismissal. Deactivating a doctor now requires confirmation.

04 Verification

Each item below was executed against a running instance. Behaviour was confirmed by observation, not by inspection of the source.

TEST	EXPECTED	OBSERVED	RESULT
HTML payload renamed <code>.jpg</code> , declared <code>image/jpeg</code>	422	422	PASS
SVG containing script	422	422	PASS
EXIF device serial in uploaded JPEG	Absent from stored object	Absent	PASS
Image request by a non-owning clinician	404	404	PASS
Image request without bearer token	401	401	PASS
Authentication with incorrect password	401	401	PASS
Repeated authentication attempts	Throttled	Throttled after 5/min	PASS
Retinal image on diagnosis result page	Renders	Renders	PASS
Exported PDF report	Watermarked, confidence as percentage	Watermarked, 86%	PASS

TEST	EXPECTED	OBSERVED	RESULT
Doctor dashboard indicators	Reflect stored data	Correct counts	PASS
Audit log after authentication events	Populated with actor and address	Populated	PASS
Layout at 375 px viewport	Drawer navigation, no page overflow	Confirmed	PASS
Schema migration, upgrade and downgrade	Both directions clean	Both clean	PASS
Browser console during session	No errors	None	PASS
Static analysis and production build	Clean	Clean	PASS

Two defects were found only by execution and would not have surfaced from reading the code: the seeding script bypassed the validation applied by the API and could create an account that authenticated successfully but whose every response failed serialisation; and the dashboard prefixed a title onto a name that already contained one. Both were corrected.

05 Model integration contract

The grading model is being developed separately. The boundary is therefore specified as a contract rather than left as internal scaffolding. Connecting a model requires implementing one class; no other part of the application changes.

```
class GradingBackend:
    name = "swin-grading"      # identifies the model
    version = "1.0.0"          # increment when weights change

    def __init__(self) -> None:
        ...                    # load the checkpoint once, not per request

    def predict(self, image_png: bytes) -> DiagnosisResult:
        ...
```

ASPECT	SPECIFICATION
Input	Validated RGB PNG bytes, EXIF removed. Not resized, normalised or colour-corrected — all preprocessing belongs to the implementer.
Output	<code>dr_stage</code> 0–4 (ICDR), <code>dr_label</code> matching the stage, <code>confidence</code> in [0, 1] or <code>None</code> , <code>is_simulated=False</code> , plus provenance strings. Ranges are enforced at construction.
Concurrency	Invoked from a thread pool, never on the event loop. May block. Must be safe for concurrent calls or guard itself.
Failure	Raise. The route converts the exception into a 503 without leaking a traceback.
Activation	Register in <code>_BACKENDS</code> and set <code>MODEL_BACKEND</code> in the environment.

Calls already run in a thread pool under the stub, where blocking is harmless. The arrangement exists so that connecting a real model — where a single inference would otherwise stall every concurrent request — requires no change to the route.

06 Known limitations

Stated explicitly so they are not mistaken for oversights. Each was a deliberate decision with a rationale.

LIMITATION	RATIONALE	STATUS
No diagnostic model connected	Owned by a separate developer and not yet available. The platform runs a stub and declares it at every surface.	BY DECISION
Session token held in <code>localStorage</code>	Readable by any script on the page. Migration to <code>httpOnly</code> cookies requires reworking the authentication flow and was not attempted. Exposure is bounded by the reduced 60-minute token lifetime and by upload sanitisation, which closes the principal injection route.	OUTSTANDING
No automated test suite	<code>pytest</code> is declared as a dependency but no tests are written. Verification for this cycle was manual and is recorded in section 04.	OUTSTANDING
Explainability module removed	It returned a path to a file that neither existed nor was served, rendering a broken image beneath an authoritative heading. It returns with the segmentation work.	DEFERRED
Dataset licensing unresolved	The research repository records no licence, ethics approval or redistribution terms for FGADR. If demonstration data originates	UNRESOLVED

LIMITATION	RATIONALE	STATUS
	there, the question is open.	

07 Outstanding operator actions

These require access to the Supabase account and cannot be performed from the codebase. Until items 1 and 2 are complete, the exposed credentials remain valid.

- **Rotate** the database password, anonymous key and service-role key, then record the new values together with a freshly generated `SECRET_KEY`.
- **Remove** the two seeded accounts carrying the known weak passwords.
- **Create** a private storage bucket named `retina-images`, with public access disabled.
- **Back up, then migrate** the production schema. The migration renames a column and existing rows reference objects in the former public bucket, which will not resolve from a private one.
- **Populate** the new configuration keys documented in `.env.example`.

OPERATING POSTURE

The platform is suitable for demonstration and for development against the model contract. It must not be used to inform patient care while `MODEL_BACKEND` remains `stub`, and the two outstanding items in section 06 should be closed before it handles any real patient record.